INDEPENDENT RESEARCH REPORT

How Strong Is the Assurance? Verification and the Limits of Independent AI Safety Evaluation

A qualitative evidence synthesis, claim-to-evidence framework, and illustrative ML experiment

Draft dated: 30 October 2026 | Status: Author-review manuscript — revisions and source checks ongoing

Author: Ishika Titar | Independent research project; no institutional affiliation claimed

Research status: Targeted, non-systematic analytical synthesis and proposed reporting framework; not peer-reviewed or experimentally validated as an AI assurance method.

Abstract

Independent evaluations can accurately assess a model under tested conditions without necessarily establishing that the deployed system has the same weights, runtime, safeguards, or configuration over time. This report examines (RQ1) methods for establishing correspondence between evaluated and deployed AI systems and (RQ2) the effect of correspondence uncertainty on the assurance justified by safety evaluations. Using a targeted, non-systematic synthesis of third-party auditing literature, model-signing specifications, technical proposals, and relevant empirical research, it organizes correspondence into five analytical layers: artifact, runtime, behavior, safeguards/configuration, and continuity. A claim-to-evidence matrix, an explicit unable-to-assess state, an evidence-ownership record, and a proposed correspondence statement make the limits of each finding explicit. As an illustrative supplement, a fixed logistic-regression classifier was evaluated at thresholds of 0.50 and 0.80 on the same test set, showing changes in precision and recall without retraining. The experiment is not an AI safety audit and cannot validate the proposed verification framework. The central conclusion is that external assessments can supply meaningful but scoped assurance; claims about deployed safeguard effectiveness require stronger correspondence evidence than claims about evaluation design alone. [1–5]

Keywords: frontier AI auditing; model identity; deployment attestation; safety assurance; evidence provenance; configuration drift

1. Introduction

A reported safety score is evidence about a specified evaluation, not an unconditional guarantee about a deployed AI service. Independent assessors may replicate test outcomes, review evaluation design, or challenge developer conclusions using separate methods. These functions

are valuable, yet they do not automatically establish the identity or behavior of the system later exposed to users. The Frontier Model Forum discusses distinct confirmation, robustness, and supplementary functions and the access each requires. [1]

A further challenge arises when model weights remain fixed but surrounding elements—system prompts, filters, routing, tools, inference settings, and monitoring rules—change. The frontier AI auditing literature describes risks from treating a staging configuration as a proxy for the operational system. [2] This report addresses the evidential connection between the system evaluated and the system deployed; it does not attempt to certify the safety of any frontier model.

2. Research questions and scope

Main question: How does uncertainty about model identity and deployment configuration affect independent assurance of frontier AI safety claims?

RQ1: What technical and procedural methods can auditors use to establish correspondence between evaluated and deployed AI systems?

RQ2: How does uncertainty about model identity and deployment configuration limit the assurance independent evaluations can provide?

Scope is restricted to evidence requirements, trust assumptions, and the kinds of claims an auditor can responsibly support. “Correspondence” means a documented and justified relationship between evaluated and deployed systems; exact bit-for-bit equality is not automatically necessary for every claim, and apparent similarity is not automatically sufficient.

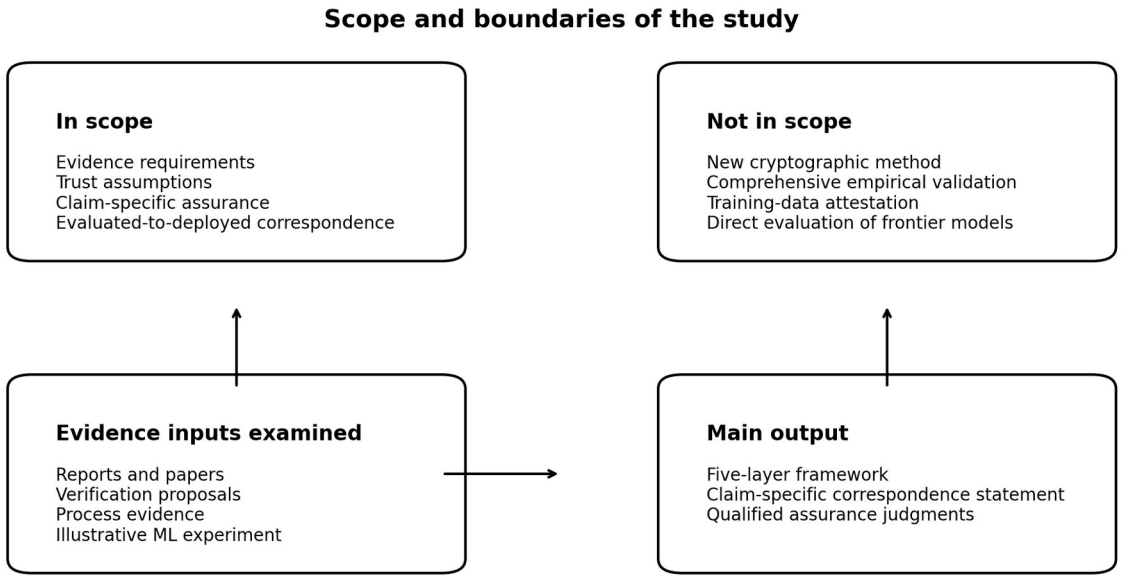


Figure 1. Scope and boundaries of the study. The paper examines evidence requirements, trust assumptions and claim-specific assurance for evaluated-to-deployed correspondence; it does not claim to introduce a new cryptographic method or empirically validate the full framework.

3. Methodology

3.1 Research design

This is a desk-based, qualitative, targeted evidence synthesis rather than a systematic literature review. Core documents were the Frontier Model Forum report, the Brundage et al. frontier auditing paper, and a proposed IETF lifecycle attestation Internet-Draft. Supplementary research was sought on model signing, substitution detection, auditing access, zero-knowledge proofs, model changes, and scope-limited audit opinions. Sources were compared by the verification question addressed, evidence required, dependence on developer-controlled information, residual uncertainty, and reported maturity. [1–12]

3.2 Search, screening and reading depth

The source notes document targeted searches and selective source reading. Full-text sections of Kim et al. (2026) were reviewed alongside its deployment and maintenance criteria; some other sources were assessed at abstract or documentation level. This account is neither a systematic search nor a comprehensive review of all maturity-matrix criteria, and should not be described as such. Source-specific uncertainty is preserved. [1–13]

3.3 Analytical process

- Extract propositions and source-specific limitations; avoid converting proposals into demonstrated deployments.
- Classify correspondence into five analytical layers and map candidate verification methods to the layers they can address.
- For each safety-claim type, identify dependent layers and the degree of support available under incomplete correspondence.
- Record trust roots, adverse assumptions, and open questions; mark framework judgments as author inference.
- Reconcile source claims with the author's RQ1/RQ2 drafts, including previously overstated claims.

3.4 Research integrity

The analytical synthesis and manuscript were developed with generative-AI assistance for literature discovery, comparison, drafting and organization. Before public release, the author must verify claims, quotations and citations and confirm the account of contributions and assistance. The proposed framework has not been independently evaluated; no deployed frontier system was audited.

4. Background: what third-party assessment establishes

Confirmation activities can include replication, methodological review, and examination of the reasoning linking evidence to conclusions. Robustness testing can challenge claimed safety boundaries by different tests; supplementary work may add specialist competence or capacity. Such activities address different epistemic questions. The label “third-party reviewed” is therefore underspecified without the method, model access, and tested claim. [1]

The literature argues that white-box or outside-the-box information may be necessary for rigorous audits, while mere access remains insufficient without sound methodology and independent judgment. [3] Auditing proposals also distinguish increasingly demanding assurance arrangements, but their assurance-level taxonomy should not be equated with the author’s own categories. [2]

4.1 Relationship to an existing third-party assurance framework

Kim et al. (2026) [13] design a third-party assurance process combining a stakeholder responsibility matrix, interview protocols, a maturity matrix and an assurance-report template. Their preliminary evaluation consisted of two retrospective applications to non-frontier ML tools and six expert interviews. The authors describe the findings as exploratory; the pilots did not establish the framework’s predictive or longitudinal effectiveness.

The comparison notes reviewed the introduction, Sections 3–6, relevant appendices and deployment/monitoring matrix rows. In those sections, the criteria focus on lifecycle processes such as release review, monitoring, updates and rollback rather than an independent cryptographic or runtime method for binding the evaluated model to the deployed service. Because not every matrix row was verified visually, this is a bounded claim about the inspected material, not an assertion of absence from the entire literature.

The frameworks are complementary in scope: Kim et al. emphasize process-and-outcome governance, while the present synthesis centers on technical and procedural correspondence at L1–L5 and claim-specific inference limits. Kim et al. explicitly account for inadequate evaluator access and role allocation, motivating the revised State 0 and layer-responsibility fields in Section 7.3. Our framework has not undergone either the preliminary pilots or the expert interviews that Kim et al. report.

5. RQ1 — Verifying correspondence

5.1 Five layers of correspondence

Layer	Verification question	Examples
L1 Artifact	Are the evaluated model artifacts identifiable and matched?	Weights, tokenizer, model files, hashes
L2 Runtime	Are those artifacts executing in the production environment?	Serving binary, hardware state, deployed container

L3 Behavior	Are observed outputs consistent with the evaluated behavior?	Spot checks, response distributions
L4 Safeguards / configuration	Are protective controls and operating settings comparable?	Prompts, filters, classifiers, tools, routing
L5 Continuity	Does the documented relationship persist after evaluation?	Updates, change logs, reassessment triggers

Table 1. Author-proposed taxonomy synthesizing sources [1–3]. Layers are analytical categories, not a validated standard.

5.2 Comparison of verification methods

Approach	What it can contribute	Residual uncertainty
Declared versions / change logs	Documentation of tested and released builds.	Developer assertions may not prove what executes.
Signed artifact manifests	Integrity and provenance of named files relative to signature/key.	Does not prove live deployment identity or safety; signer trust matters.
Hardware/runtime attestation	Evidence about measured execution environment under specified trust assumptions.	Depends on measured scope and trustworthy reference values.
Black-box spot checks	Detect some observable divergences in outputs.	May miss subtle substitutions and suffer nondeterminism.
Safeguard/configuration review	Document which controls evaluation relied on.	Documents may show intended rather than actual operation.
Continuous monitoring/reassessment	Detect some changes and scope conclusions to a validity window.	Depends on monitoring coverage and change disclosure.
Cryptographic inference proofs	Can prove specified committed computation when applicable.	Does not establish semantic safety or end-to-end deployment equivalence.
Black-box spot checks / statistical signatures	Auditor-controlled probes can detect some visible substitution or behavioral divergence.	May miss subtle substitution, endpoint routing, or unobserved safeguards; not proof of exact identity.
Signed lifecycle attestation chain (proposal)	A proposed linkage among identity assertions, artifacts, deployment and inference outputs.	Individual Internet-Draft, not adopted standard; depends on key authorities, payload scope, implementation and external verification.

Table 2. Synthesis of candidate methods, not a field-tested ranking. Sources include [1–4, 7–9, 12].

The reviewed methods provide complementary evidence. An artifact signature can establish integrity of a referenced object, but does not alone identify what a public API serves. Hardware attestations can narrow this gap for what is actually measured; black-box tests can detect behavioral divergence but may not reveal its cause. A technical lifecycle-attestation Internet-Draft proposes chained records; it is a work in progress and should not be described as a deployed or approved IETF standard. [4, 7–9]

5.3 Provisional answer to RQ1

Auditors can build layered correspondence evidence through authenticated artifacts, measured runtime states, independent behavioral checks, configuration review, and continuing reassessment. None of the reviewed materials, taken alone, establishes complete developer-independent correspondence for a frontier deployment. This is a limitation of the reviewed evidence, not proof that such correspondence is impossible. The confidence justified by each

method also depends on who controls signing keys, reference builds, test access, and deployment records. [2–4, 7–9]

6. RQ2 — What uncertainty does to assurance

6.1 Claim-specific dependency map

The following matrix is a proposed analytical dependency map, not an empirically validated checklist. “Required” denotes evidence normally needed to justify the specified deployed-system claim; “Conditional” depends on the evaluation configuration and claim wording. The map should be interpreted alongside the original study’s uncertainty and access limitations [author synthesis; informed by Brundage et al. and FMF].

Claim	L1 artifact	L2 runtime	L3 behavior	L4 safeguards	L5 continuity
Evaluation protocol quality	Not necessary	Not necessary	Not necessary	Conditional	Not necessary
Capability claim about deployed system	Required	Conditional	Conditional	Conditional	Required if changed
Effectiveness of deployed safeguards	Required	Required	Helpful	Required	Required
Continuing safety claim	Required	Required	Helpful	Required	Required

Table 3A. Proposed claim-to-layer dependency map. Entries are author judgments, not independently validated coverage scores.

Identity and configuration uncertainty changes the scope of warranted conclusions rather than making all third-party assessment worthless. An auditor may still assess the clarity of evaluation objectives, methodology, calculations, and logic given adequate access. But a claim that deployed safeguards are effective additionally depends on whether those safeguards were present and operating in production, and whether relevant changes have occurred. This distinction is a conceptual inference combining the source literature, not a directly measured effect size. [1–3]

Claim type	Evidence most relevant	Cautious conclusion
Evaluation design validity	Protocol, rubrics, data provenance, scoring.	Can be critically reviewed without proving deployed-system identity.
Capability bound	Representative tests plus justified transfer relationship.	Cannot assume monotonic transfer across configurations.
Safeguard effectiveness	Correspondence of live controls plus targeted testing.	Weak if controls/configuration are only developer-described.
Continuing deployment safety	Configuration evidence, monitoring, changes and expiry.	Point-in-time tests do not establish lasting safety.

Table 3. Claim-specific dependence assessment; classifications are interpretive, not empirically calibrated.

Prior research reports that even small fine-tuning changes may affect alignment and that hosted LLM behavior has varied across observation periods; these illustrate sensitivity to changes but do

not measure how often independently audited safety results fail to transfer to production. [5, 6] The analogy to qualified audit opinions in financial auditing is useful only as a communication device and is not an applicable AI auditing standard. [10]

Evidence from an evaluated system should not be transferred to a deployed system solely on an assumed ordering of model capability or safeguard strength. A conservative-direction argument is defensible only if its assumptions are specified and supported for the particular claim, task, controls and conditions. The draft's proposed transfer rule is therefore treated here as a hypothesis, not a general result [author inference].

7. Proposed contribution — claim-specific correspondence statement

This report proposes a structured, claim-specific "correspondence statement" linking conclusions to system identity evidence, deployment dependencies and unassessed conditions. It draws from prior auditing and assurance research; neither methodological priority nor operational effectiveness is claimed. [2, 3, 13]

Field	What a report should record
System evaluated	Model identifier, configuration, safeguards, date and deployment pathway.
Verified layers	For L1–L5: evidence, method, verification result and trust root.
Unverified layers	Exclusions, denied access, uncertainty and rationale.
Relationship to deployment	Whether and how correspondence is established; assumptions.
Supported claims	Precisely bounded claim and why evidence supports it.
Unsupported claims	Claims not justified by present evidence.
Validity and review triggers	Expiry date, relevant updates, incidents and re-audit conditions.
Access state	State 0 = unable to assess; other evidence levels must identify basis and uncertainty.
Evidence responsibilities	Identify layer controller, evidence provider and independent checks by assessor.
Evidence capture	Timestamp, provenance and contemporaneous snapshots of tested and deployed configurations.

Table 4. Proposed reporting framework (author synthesis).

7.1 Illustrative application

Hypothetical case: an external assessor reviews the design of a safety evaluation and receives signed hashes of evaluated weights, but is denied independent runtime and safeguard access. Under this framework, the assessor can describe the reviewed evaluation and artifact evidence, while withholding a conclusion that deployed safeguards match or remain effective. The example is deliberately fictional; it is not a real-world audited finding.

7.2 Adversarial considerations

The framework helps state limitations but does not eliminate them. Valid signatures may refer to the wrong authorized artifact; an attested container may omit unmeasured external services; output spot checks may fail to detect targeted substitutions; monitoring can miss undisclosed changes. Consequently, a correspondence statement is a transparency aid, not a certification system.

7.3 Revised framework: access states, responsibility and evidence timing

Drawing on the comparison with Kim et al. [13], the proposed statement should include an explicit State 0 ("unable to assess") for each in-scope layer or claim where evidence is absent, access is denied, or evidence is insufficient. State 0 is not a claim that the system is unsafe; it means the assessor cannot draw the specified conclusion. A complete report should enumerate unsupported claims to avoid selective presentation of favorable findings.

Each layer should identify (a) the party controlling the relevant artifact, runtime, endpoint, safeguard or monitoring process; (b) who supplied the evidence; (c) what the independent assessor actually inspected; and (d) the trust dependencies that remain. Responsibilities may be split between model developers, cloud hosts, integrators and application builders. Documentation about update approvals can provide process evidence for L5 but is not a technical proof that the inspected version is currently deployed.

Evidence should be captured when the evaluation occurs: signed artifact manifests, deployment configuration snapshots, assessor test queries, reviewed safeguard settings and dated change records. Reporting should state when evidence was collected and the changes that would trigger reassessment. These additions are design proposals informed by [13], not empirically verified improvements.

The proposed correspondence statement should also require an exhaustive list of in-scope claims, including unsupported ones; an explicit "unable to assess" outcome where evidence or access is insufficient; and a named evidence supplier, layer controller, verification action, and remaining trust dependency for each layer. This avoids presenting partial evidence as comprehensive verification. Process records and change approvals may inform L4 or L5, but do not independently establish live correspondence. These are reporting design proposals, not empirically tested improvements [author synthesis informed by Kim et al., 2026].

8. Illustrative ML experiment

8.1 Aim and setup

A separate synthetic binary-classification demonstration examined whether altering an operational decision rule changes measured outcomes while preserving the trained model. A logistic regression model was trained once on a synthetic dataset of 1,500 observations and 12 features; 70% of observations were used for training and 30% for testing (450 observations). The

same held-out test set and learned model were evaluated with decision thresholds 0.50 and 0.80. The local Antigravity terminal screenshot shows a successful run and a results file saved. The exact source code, package versions, and random-state settings must be checked against the final repository before publication.

Metric	Threshold 0.50	Threshold 0.80
Accuracy	0.831	0.773
Precision	0.794	0.974
Recall	0.612	0.273
False positives	22	1
False negatives	54	101

Table 5. Output observed in the provided Antigravity terminal screenshot, 30 October 2026; rounded to three decimals.

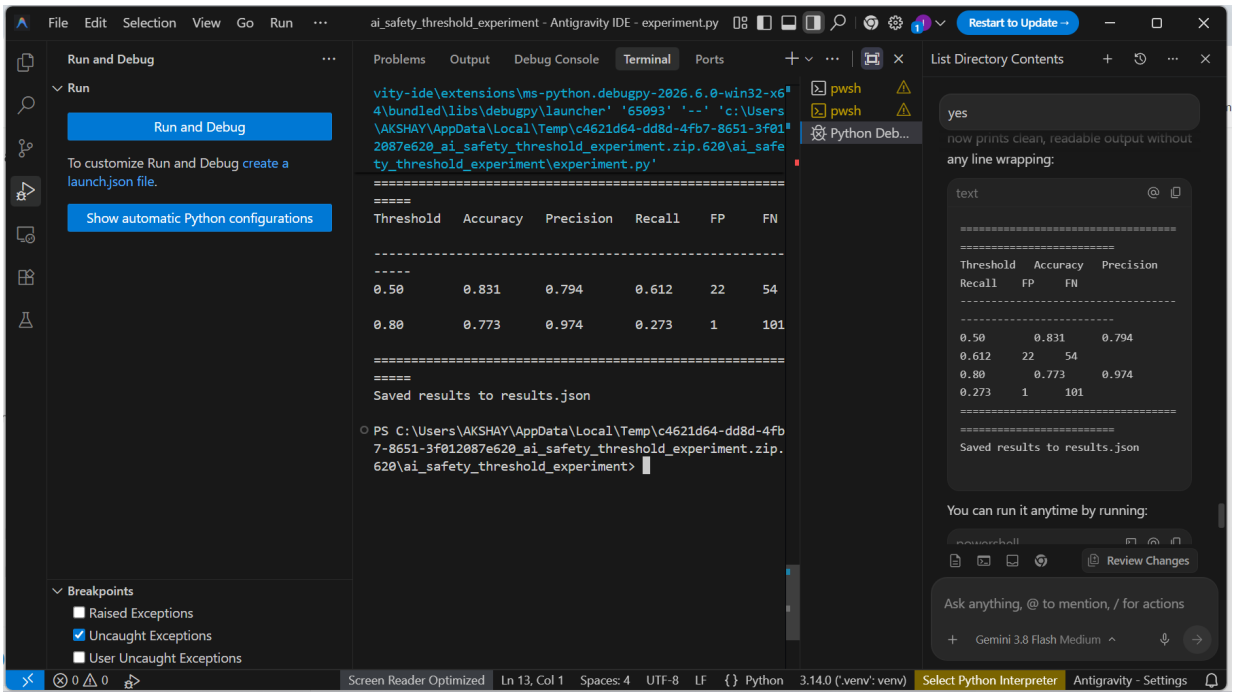


Figure 2. Successful local run of the synthetic classifier experiment in Antigravity, showing the two decision thresholds and saved results.

8.2 Interpretation

Raising the decision threshold yielded fewer positive classifications: false positives decreased from 22 to 1, while false negatives increased from 54 to 101. Precision increased and recall fell. The observed tradeoff demonstrates that keeping a trained classifier fixed does not guarantee identical decisions if a downstream threshold changes. In this experiment, the positive class is synthetic and not directly a category of harmful AI behavior.

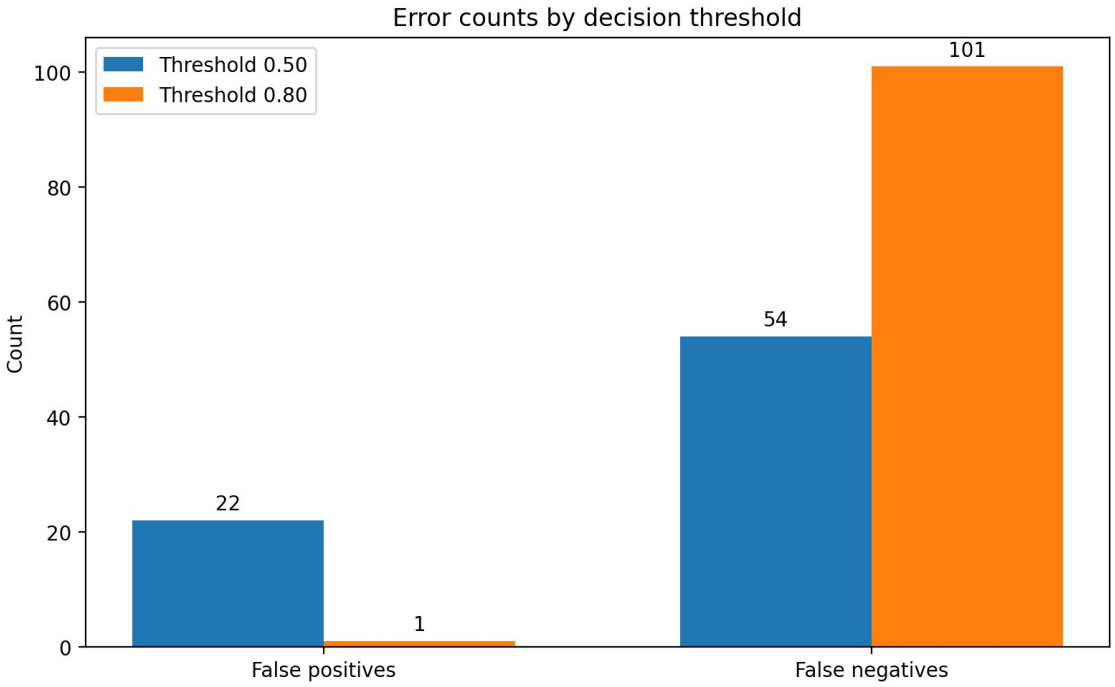


Figure 3. Error counts by decision threshold. Raising the threshold from 0.50 to 0.80 reduced false positives from 22 to 1 but increased false negatives from 54 to 101.

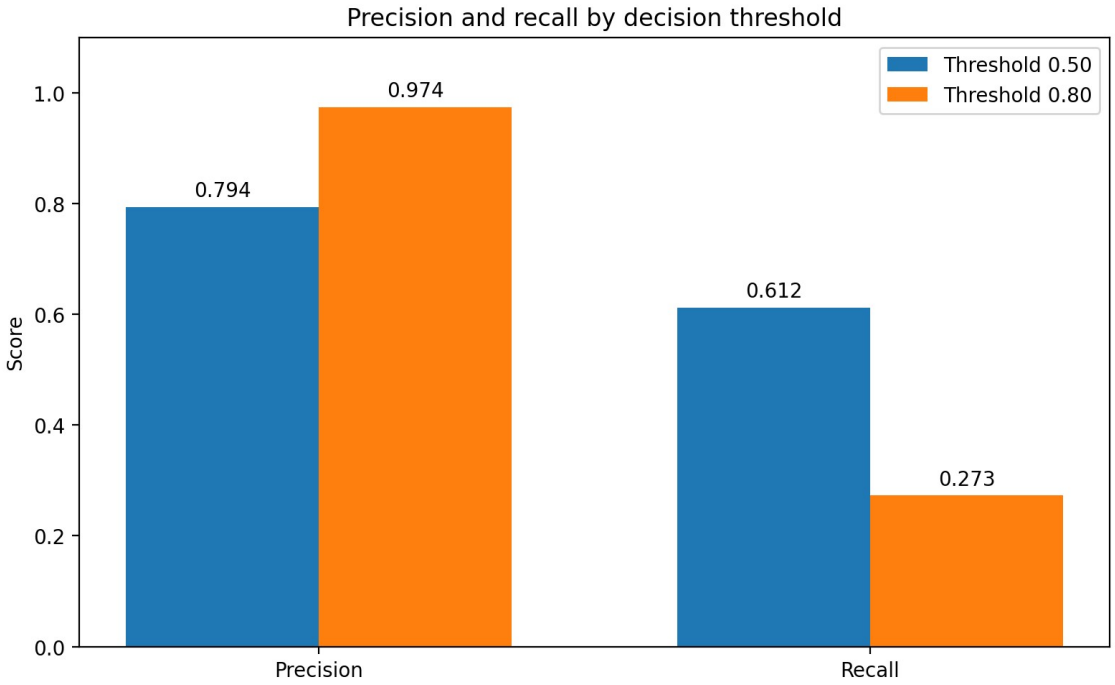


Figure 4. Precision and recall by decision threshold. The higher threshold increased precision (0.794 to 0.974) while reducing recall (0.612 to 0.273).

8.3 Scope and limitations

The demonstration uses an ordinary supervised classifier, not a frontier language model, an adversarial benchmark, or an actual production system. It therefore supports only a narrow illustrative premise about configuration sensitivity and cannot validate the five-layer taxonomy, establish cryptographic identity, prove an AI security vulnerability, or estimate real-world safety risk. No additional runs, sensitivity analysis, confidence intervals, or empirical assessment of the correspondence framework are claimed.

9. Discussion

The combined RQ1/RQ2 analysis motivates a change from binary labels such as “externally audited” to clearly bounded claim-and-evidence statements. The central inference is that verification should track multiple, sometimes independently controlled layers; assurance depends on which layers matter for a particular claim and what evidence has actually been examined. This approach can make residual uncertainty visible without treating all incomplete audits as worthless. Its practical value remains untested.

Two cautions follow. First, verification of a computation or artifact does not establish that an evaluation targeted the appropriate harm or that a system is globally safe. Second, differences between tested and deployed systems do not automatically invalidate every earlier finding: their importance depends on the specific claim and an empirically or logically justified relationship between configurations. [1–4]

10. Limitations and future work

The Kim et al. comparison was based on specific full-text sections and selected matrix rows; a final row-by-row visual check was not completed. Accordingly, the related-work claim is restricted to the inspected sections. In addition, the proposed access states and reporting fields are design suggestions, not validated performance measurements.

- Targeted, non-systematic reading may have missed important prior work, limiting originality claims.
- Several technically consequential references were evaluated only through abstracts or proposal text.
- Layer coverage, independence, and assurance judgments are qualitative and were not independently rated.
- No developer deployment was accessed, no live-model correspondence was measured, and no framework effectiveness study was conducted.
- The classifier demonstration is illustrative and has no causal connection to frontier-model assurance effectiveness.
- Future research could test whether correspondence statements improve auditor/user interpretation and examine independently controlled reference builds and runtime verification.

11. Conclusion

External AI safety evaluations can provide meaningful evidence, but a conclusion about deployed systems requires careful attention to model identity, runtime execution, safeguard configuration, and change over time. This study offers a claim-specific correspondence taxonomy and a proposed disclosure statement to distinguish verified facts, assumptions, and unverified deployment dependencies. The contribution is an analytical synthesis and reporting proposal, not a proven deployment-verification solution. A small reproducible classifier exercise illustrates that unchanged trained parameters can coexist with changed evaluation outcomes under a different decision threshold. The most defensible audit conclusion is therefore scoped to the evidence, model conditions, and period actually examined.

References

[1] Frontier Model Forum. “Third-Party Assessments.” Technical report, 4 August 2025. https://www.frontiermodelforum.org/technical-reports/third-party-assessments/

[2] Brundage, M., et al. “Frontier AI Auditing: Toward Rigorous Third-Party Assessment of Safety and Security Practices at Leading AI Companies.” arXiv:2601.11699 (2026). https://arxiv.org/abs/2601.11699

[3] Casper, S., et al. “Black-Box Access Is Insufficient for Rigorous AI Audits.” ACM FAccT (2024). https://doi.org/10.1145/3630106.3659037

[4] Sharif, Raza. “Cryptographic Attestation for AI Model Lifecycle: From Training Data to Inference Output.” Individual Internet-Draft draft-sharif-ai-model-lifecycle-attestation-01, 26 August 2026 (work in progress; not an IETF-approved standard). https://datatracker.ietf.org/doc/html/draft-sharif-ai-model-lifecycle-attestation-01

[5] Qi, X., et al. “Fine-tuning Aligned Language Models Compromises Safety, Even When Users Do Not Intend To!” ICLR (2024). https://arxiv.org/abs/2310.03693

[6] Chen, Lingjiao, Matei Zaharia, and James Zou. “How Is ChatGPT’s Behavior Changing over Time?” arXiv:2307.09009 (2023). https://arxiv.org/abs/2307.09009

[7] Open Source Security Foundation (OpenSSF). “OpenSSF Model Signing (OMS).” Project documentation (accessed October 2026). https://openssf.org/projects/model-signing/

[8] Cai, W., et al. “Are You Getting What You Pay For? Auditing Model Substitution in LLM APIs.” arXiv:2504.04715 (2025). https://arxiv.org/abs/2504.04715

[9] Sun, Haochen, Jason Li, and Hongyang Zhang. “zkLLM: Zero Knowledge Proofs for Large Language Models.” arXiv:2404.16109 (2024). https://arxiv.org/abs/2404.16109

[10] IAASB. ISA 705 (Revised): Modifications to the Opinion in the Independent Auditor’s Report. Used only for analogy; not an AI auditing requirement. [Bibliographic URL and edition not yet independently verified.]

[11] ICAEW Audit and Assurance Faculty. “Limitation on the Scope of the Audit” (2021). Used only as a contextual analogy. [Exact publication details and URL still require verification.]

[12] Fortanix. “AI Attestation” FAQ / vendor documentation. Claims require independent technical corroboration. [Precise source page/version still requires verification; vendor claims are not independent evidence.]

[13] Kim, R. M., Kuehnert, B., Lai, A., Holstein, K., Heidari, H., and Ghani, R. “Toward Third-Party Assurance of AI Systems: Design Requirements, Prototype, and Early Testing.” ACM FAccT (2026). https://arxiv.org/abs/2601.22424 ; https://doi.org/10.1145/3805689.3806759